

Phase A Implementation Plan - SIS Development

Overview

Complete Phase A implementation focusing on development efficiency since teachers won't adopt until feature-complete. Building foundation → gradebook → attendance → scheduling → integration.

Phase A.1: Complete Foundation Structure

Timeline: Week 1-2 Goal: Fix existing foundation + add missing core academic models

Week 1: School Years & Subjects

Backend Tasks:

- ☐ Fix school_years model (what you started)
- ☐ Create/update migration for school_years table
- ☐ Fields: id, name (e.g., "2024-2025"), start_date, end_date, is_active, school_id
- ☐ Add school_year context to user sessions
- ☐ Create subjects model
- ☐ Fields: id, name, code, type (core/special), grade_levels, school_id
- ☐ Seed data: Math, Reading, Science, Art, PE, Music, etc.
- ☐ API endpoints: CRUD for subjects

Frontend Tasks:

- ☐ Add school year selector to admin panel
- ☐ Subject management UI in admin panel
- ☐ Update header to show active school year context

Week 2: Classrooms & Enrollments

Backend Tasks:

- ☐ Create classroom/section model
- ☐ Fields: id, name, subject_id, teacher_user_id, school_year_id, grade_level, room_number
- ☐ Examples: "Mrs. Smith's 3rd Grade", "7th Grade Math - Period 2"
- ☐ Update enrollment model
- ☐ Change from student→teacher to student→classroom_section
- ☐ Fields: id, student_id, classroom_id, enrollment_date, status
- ☐ Update existing APIs to use classroom-based assignments

Frontend Tasks:

- ☐ Classroom creation UI for admin
- ☐ Student enrollment to classrooms (not direct to teachers)
- ☐ Teacher dashboard showing their classrooms

- ☐ Update admin student assignment to use classrooms
-

Phase A.2: Gradebook System

Timeline: Week 3-5 Goal: Complete gradebook with categories, assignments, grades, and calculations

Week 3: Gradebook Data Models

Backend Tasks:

- ☐ Create gradebook_categories model
- ☐ Fields: id, classroom_id, name, weight, drop_lowest_count, default_category
- ☐ Examples: "Homework" (weight=1), "Tests" (weight=2), "Projects" (weight=1.5)
- ☐ Create assignments model
- ☐ Fields: id, classroom_id, category_id, title, description, max_points, due_date, weight_override, late_policy
- ☐ Create grades model
- ☐ Fields: id, assignment_id, student_id, points_earned, status, late_penalty, comments, graded_by, graded_at
- ☐ Status enum: submitted, missing, late, excused, in_progress

Database:

- ☐ Create migrations for all gradebook tables
- ☐ Add indexes for performance (student_id, assignment_id, classroom_id)

Week 4: Gradebook Business Logic

Backend Tasks:

- ☐ Grade calculation engine
- ☐ Category-weighted calculations
- ☐ Drop-lowest logic per category
- ☐ Handle missing vs excused assignments
- ☐ Real-time calculation triggers
- ☐ Assignment CRUD APIs
- ☐ Create assignment with auto-student linking
- ☐ Bulk assignment creation
- ☐ Assignment templates/copying
- ☐ Grade entry APIs
- ☐ Individual grade updates
- ☐ Bulk grade import (CSV/paste)
- ☐ Grade history/audit logging

Week 5: Gradebook UI

Frontend Tasks:

- ☐ Teacher gradebook interface
 - ☐ Spreadsheet-like grade entry grid
 - ☐ Category management (create, edit, reorder)
 - ☐ Assignment creation with inline category creation
 - ☐ Real-time grade calculations display
 - ☐ Admin gradebook oversight
 - ☐ View all classrooms' gradebooks
 - ☐ Grade distribution reports
 - ☐ Student/parent read-only views
 - ☐ Individual student grade summary
 - ☐ Category breakdowns and assignment details
-

Phase A.3: Attendance System

Timeline: Week 6 Goal: Complete attendance tracking with calendar integration prep

Week 6: Attendance Implementation

Backend Tasks:

- ☐ Create attendance_records model
- ☐ Fields: id, student_id, classroom_id, date, status, reason, notes, recorded_by, recorded_at
- ☐ Status enum: present, absent_excused, absent_unexcused, tardy_excused, tardy_unexcused
- ☐ Attendance business logic
- ☐ Auto-default to "present" if not marked within time window
- ☐ Bulk attendance entry for classroom
- ☐ Attendance correction/history
- ☐ Attendance reporting APIs
- ☐ By student, classroom, date range
- ☐ Chronic absence alerts
- ☐ Attendance trends and analytics

Frontend Tasks:

- ☐ Teacher attendance interface
- ☐ Daily classroom roster with quick status buttons
- ☐ Attendance history view
- ☐ Missing attendance alerts
- ☐ Admin attendance reports
- ☐ School-wide attendance dashboards
- ☐ Chronic absence tracking
- ☐ Parent attendance view
- ☐ Child's attendance history and trends

Phase A.4: Scheduling System

Timeline: Week 7-8 Goal: Complete time-based scheduling with conflict detection

Week 7: Scheduling Foundation

Backend Tasks:

- ☐ Create calendar_blocks model
- ☐ Fields: id, classroom_id, day_of_week, start_time, end_time, weekly_minutes, is_special_subject
- ☐ Create specialist_requests model
- ☐ Fields: id, specialist_user_id, student_id, requested_time, status, approved_by, reason
- ☐ Scheduling business logic
- ☐ Conflict detection (teacher/room/student overlaps)
- ☐ K-5 vs 6-8 scheduling rules
- ☐ Weekly minute tracking vs requirements

Week 8: Scheduling UI & Specialist Integration

Frontend Tasks:

- ☐ Admin schedule builder
 - ☐ Calendar view with drag-drop time blocks
 - ☐ Conflict visualization and resolution
 - ☐ K-5 flexible vs 6-8 structured modes
 - ☐ Specialist pull-out workflow
 - ☐ Specialist request interface
 - ☐ Teacher approval/denial with alternatives
 - ☐ Pull-out conflict detection
 - ☐ Teacher schedule view
 - ☐ Daily/weekly schedule display
 - ☐ Integration with attendance (click block → attendance)
-

Phase A.5: Integration & Polish

Timeline: Week 9-10 Goal: Connect all systems and meet Phase A requirements

Week 9: Cross-System Integration

Backend Tasks:

- ☐ Integrate scheduling with attendance
- ☐ Click calendar block → attendance sheet
- ☐ Attendance linked to specific class periods
- ☐ Integrate scheduling with gradebook
- ☐ Assignments can be linked to calendar blocks

- ☐ Grade entry from schedule view
- ☐ Performance optimization
- ☐ Database query optimization
- ☐ Caching for teacher dashboards
- ☐ Real-time updates where needed

Frontend Tasks:

- ☐ Unified teacher workflow
- ☐ Schedule → Attendance → Gradebook flow
- ☐ Context switching between classrooms
- ☐ Admin analytics dashboard
- ☐ School-wide performance metrics
- ☐ Teacher workload analysis
- ☐ Student performance correlations

Week 10: Phase A Requirements Compliance

Validation Tasks:

- ☐ **Performance Benchmarks:**
 - ☐ Attendance completion: ≤ 2 min/class (p50), ≤ 4 min (p95)
 - ☐ Grade entry: 25 students in ≤ 5 minutes
 - ☐ Specialist approval: ≤ 24 hour turnaround median
 - ☐ Schedule conflicts: $\geq 99\%$ resolved before publish
 - ☐ **Feature Completeness:**
 - ☐ All Phase A requirements from BRD implemented
 - ☐ Role-based permissions working correctly
 - ☐ Parent/student read-only access functional
 - ☐ **Preparation for Phase B:**
 - ☐ Data models support lesson planning integration
 - ☐ APIs ready for curriculum mapping
 - ☐ Assignment creation ready for lesson linking
-

Success Criteria

Technical Metrics

- ☐ All Phase A BRD requirements implemented
- ☐ Performance benchmarks met
- ☐ 99%+ uptime during testing
- ☐ Role-based security validated

User Experience

- ☐ Admin can manage complete school structure

- ☐ Teachers can efficiently manage grades and attendance
- ☐ Parents/students have read-only access to relevant data
- ☐ Intuitive workflows for daily teacher tasks

Data Integrity

- ☐ Grade calculations accurate and real-time
 - ☐ Attendance defaulting and tracking working
 - ☐ Schedule conflicts properly detected and resolved
 - ☐ Audit trails for all major data changes
-

Key Dependencies & Assumptions

Dependencies

- Current user/school/role system remains stable
- Database performance adequate for real-time calculations
- Frontend framework handles spreadsheet-like interfaces well

Assumptions

- Teachers won't use system until feature-complete
- Phase B (lesson planning) will build on Phase A foundation
- Specialist pull-out scheduling can be simplified for MVP

Risk Mitigation

- Build gradebook calculation engine early (most complex)
- Test performance with realistic data volumes
- Validate workflows with actual teacher feedback before launch